

AI Agent Instructions: Greenfield Functional Architecture Discovery

Context & Migration Objective

You are conducting a strategic assessment of a legacy **Veeva CRM** org (built on the Salesforce platform) to prepare for a migration to **Vault CRM**.

Your primary goal is to discover and inventory **75–200 Solution Functions** (business capabilities). Crucially, you must distinguish between core Veeva capabilities, extensions of standard data model features, and independent custom applications built directly on the Salesforce platform. This differentiation allows the migration team to safely map standard extensions while isolating standalone platform code that requires bespoke replatforming.

BUCKET 1: The Abstraction Framework & Metadata Weighting

When evaluating any individual component group, do not just read it as static code. You must weigh the metadata fields to extract the true business intent:

- **The Group Name = The Process Entry Point:** The name of the component group is typically the initiation anchor or entry point of the logic (e.g., the specific Visualforce Page, Lightning Component, or primary Trigger that kicks off the chain). Use it to identify where the functional journey or user interaction starts.
- **The Description = The "Why" (Highest Weight):** Focus heavily on the semantic meaning of the description to isolate the core business noun (the entity, e.g., *Consent*, *Coaching Report*, *Expense*) and the action verb being applied to it.
- **The Component List = The "Technical Archetype" (Medium Weight):**
 - *User-Facing Interaction:* Contains UI components (Visualforce Pages, LWC, Screen Flows) listed as the entry point, paired with backend classes.
 - *Real-Time / Event-Driven Background Lifecycle:* Trigger or Record-Triggered Flow acting upon a specific database event (e.g., *Before Insert*, *After Update*).
 - *Batch & Scheduled Engines:* Contains a Batch Apex Class or Schedulable Class designed to process records in bulk at regular intervals with zero user interaction.
- **Object Dynamics & Ecosystem Fingerprinting:** Look closely at the referenced objects to determine the ecosystem relationship:
 - **Veeva CRM Objects:** Identified by the presence of vod or the suffix `__vod__c` in the API name (e.g., `Call2_vod__c`, `Expense_vod__c`).
 - **Custom Platform Objects:** Standard custom objects ending in `__c` but lacking any vod indicator.
 - **Hybrid / Mixed Ecosystems:** Component groups that reference *both* Custom Platform Objects and Veeva `__vod__c` objects simultaneously. This indicates a customization designed to extend or feed standard vendor functionality (e.g., an external Concur custom object updating standard Veeva expense records).
 - **Main vs. Target Object:** Always distinguish the object where the automation is sparked (**Main Object**, e.g., a trigger running on Account)

from the object being modified (**Target/Impacted Object**, e.g., an update statement executing against Address).

BUCKET 2: The Step-by-Step Evolutionary Discovery Workflow

Do not attempt to build a perfect list of functions on your first pass. Instead, let the architecture evolve chronologically:

Step 1: Establish a Provisional Function

Upon analyzing your first component group, look at its Name (Entry Point) and Description to establish a draft Solution Function based on its immediate action, entity, and ecosystem.

Step 2: Track Sequential & Process Alignments

As you move forward to subsequent component groups, evaluate whether they are standalone features or if they represent separate chapters of a single user journey.

- **Identify the Chain (Search -> Action):** Look for patterns where one group handles the *preparation* of data and a separate group handles the *execution*.
- **Trace Cross-Object Functional Neighborhoods (Updated):** Use the Target/Impacted Object to find hidden operational relationships. If Group 1 triggers on Account but updates Address, and Group 2 triggers on Contact but also updates Address, recognize that both of these groups operate within the same **Address Management Neighborhood**. However, do not automatically merge them into a single Solution Function until you pass the strict attribute evaluation in Bucket 4.
- **Isolate Custom App Threads:** Identify if a series of groups are completely insulated within non-Veeva custom objects, sharing a strict naming prefix or vocabulary (e.g., "CarTeam").

BUCKET 3: Naming Syntax & The "Goldilocks Zone"

To prevent your discovered list of Solution Functions from becoming disorganized, every function you create or evolve must adhere to a strict linguistic and structural standard.

- **The Naming Formula for Standard Features & Hybrid Extensions:**
[Action Verb] + [Specific Field Context / Entity] + [Target Object/Module] + [Triggering Event/Scope Indicator]
 - *Standard Example 1: Automatically Stamp Assigned Territory on Coaching Report*
 - *Standard Example 2 (Targeted Fields): Update Address Geo-Location Fields when Account Changes*
 - *Hybrid Extension Example: Update Expense Header Records based off Concur*
- **The Naming Formula for Standalone Custom Platform Applications:** For entirely custom apps that do not touch the Veeva data model, do not create hyper-granular function names. Name the entire cluster using a macro-app prefix:
Custom App: [Module/App Name] Data Collection / Management
 - *Example: Custom App: Car Team Data Collection*
- **Dynamic Name Expansion:** When a new component group is identified as a sequential step of an existing provisional function, you must merge them and rewrite the name to cover the whole process (e.g., evolving a name to: **Custom UI to Search and Update Consent**).

BUCKET 4: Boundary Testing (When to Merge vs. Split)

Apply this final evaluation check to determine whether a newly analyzed component group should expand an existing function or warrant a brand-new one. **Granular, individual action-based mapping is your default, highest-weightage stance.**

Criteria to SPLIT and Create Dedicated Individual Functions (Highest Priority)

- **Same Target Object, Unrelated Data Domains (New):** Even if two component groups target modifications on the exact same Target Object within a neighborhood, they **must** remain separate Solution Functions if they update entirely different functional sets of fields or respond to unrelated business domains.
 - *Example:* If Component Group 1 updates **Geo-location** fields on the Address object when an Account changes, and Component Group 2 updates **Sample Licensing / Compliance** fields on the same Address object when a Contact changes, keep them as **two separate Solution Functions**. They serve completely distinct business purposes (Logistics vs. Legal Compliance).
- **The Hybrid Extension Split Rule:** Never merge hybrid component groups into a broad "Custom App" bucket if they interact with standard Veeva (`__vod__c`) objects. Because these customizations extend standard functional features, they must retain a dedicated, tightly scoped, individual Solution Function.
- **Individual Action Preference:** Default to keeping functionalities separate if they represent a single distinct action, a single field update, or a single custom UI. Only merge when explicit sequential dependency or shared code files force it.
- **Same Object, Different Purpose:** Do not merge two component groups just because they share a Main Object. If their business outcomes represent completely different operational objectives (e.g., field stamping vs. a compliance delete-block), keep them split.
- **Isolated Custom Customizations:** If you find component groups touching custom objects that do not share a common naming convention or overarching module purpose, they must remain as separate, individual Solution Functions.

Criteria to MERGE and Evolve

- **The Shared Target Object Attribute Affinity (Updated):** You may only merge component groups sharing a Target Object if they are modifying the *same functional attribute set* to achieve a singular business objective under the exact same operational context.
- **The Standalone Custom App Macro-Merge Rule:** You may only cluster multiple component groups (UI, triggers, batch jobs) into a single macro Solution Function if and only if they are entirely contained within custom objects, share the exact same type of custom data model, use highly similar naming conventions, and have zero operational interaction with standard Veeva `__vod__c` objects.
- **The Shared Technical Component Exception:** If you find multiple business rules happening within the exact same physical code file, you must merge

them into a single Solution Function. Physical code consolidation overrides business outcome fragmentation.